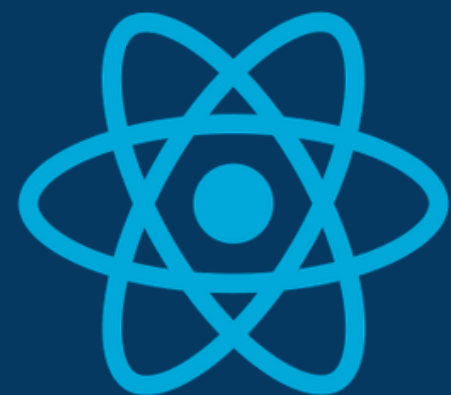


Aula 2

Desenvolvimento Mobile



React Native

PROF: MARCELO SANTOS

@professormarceloTI



ambiente de desenvolvimento TypeScript



ambiente de desenvolvimento TypeScript



TypeScript é um superset do JavaScript, o que significa que todo código JavaScript válido também é código TypeScript válido.

A diferença é que o TypeScript adiciona um sistema de tipagem estática à linguagem, permitindo definir explicitamente o tipo de dado que uma variável, parâmetro ou retorno de função deve ter.

Enquanto no JavaScript uma variável pode receber qualquer tipo de valor sem restrições, no TypeScript é possível definir desde o início qual tipo de dado é esperado, e o próprio código sinaliza um erro caso essa regra seja quebrada, antes mesmo do programa ser executado

ambiente de desenvolvimento TypeScript

Exemplo em JavaScript, sem tipagem:

```
function soma(a, b) {  
  return a + b;  
}
```

`soma(5, "3");` // não gera erro, mas o resultado é "53" (concatenação, não soma)

O mesmo exemplo em TypeScript, com tipagem:

```
function soma(a: number, b: number): number {  
  return a + b;  
}
```

`soma(5, "3");` // erro detectado antes mesmo de rodar o código

ambiente de desenvolvimento TypeScript

```
JS Aula1.js X
JS Aula1.js > ...
1 function soma(a, b) {
2   return a + b;
3 }
4 soma(5, "3");
```

```
TS Aula1.ts 1 X
TS Aula1.ts > ...
1 function soma(a: number, b: number): number {
2   return a + b
3 }
4
5 soma(5, "3")
```

Argument of type 'string' is not assignable to parameter of type 'number'.

View Problem (Alt+F8) Quick Fix... (Ctrl+.) Fix (Ctrl+I)

Por que usar TypeScript:

Menos erros: como os tipos são verificados antes da execução do código, muitos erros comuns (como somar um número com um texto por engano) são identificados ainda durante o desenvolvimento, e não apenas quando o programa já está rodando.

ambiente de desenvolvimento TypeScript



Autocomplete melhor: como o editor de código sabe exatamente qual tipo de dado cada variável ou função espera, ele consegue sugerir propriedades, métodos e parâmetros corretos automaticamente, agilizando a escrita do código.



ambiente de desenvolvimento TypeScript

Código mais seguro e documentado: a tipagem funciona também como uma forma de documentação, deixando claro, apenas lendo a assinatura de uma função, quais tipos de dados ela espera receber e o que ela retorna.



ambiente de desenvolvimento TypeScript

o TypeScript não substitui o JavaScript, ele é executado em cima dele, sendo convertido (compilado) para JavaScript puro antes de rodar no navegador ou no dispositivo.

```
TS Aula1.ts X
TS Aula1.ts > [e] mensagem
1  const mensagem: string = "Olá, TypeScript!"
2  console.log(mensagem)
3
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\marce\Desktop\Js> node Aula1.ts
Olá, TypeScript!
PS C:\Users\marce\Desktop\Js> 
```



Tipos básicos em TypeScript

O TypeScript permite definir explicitamente o tipo de dado que uma variável, parâmetro ou retorno de função deve armazenar, um recurso conhecido como tipagem estática

ambiente de desenvolvimento TypeScript



Tipos básicos em TypeScript

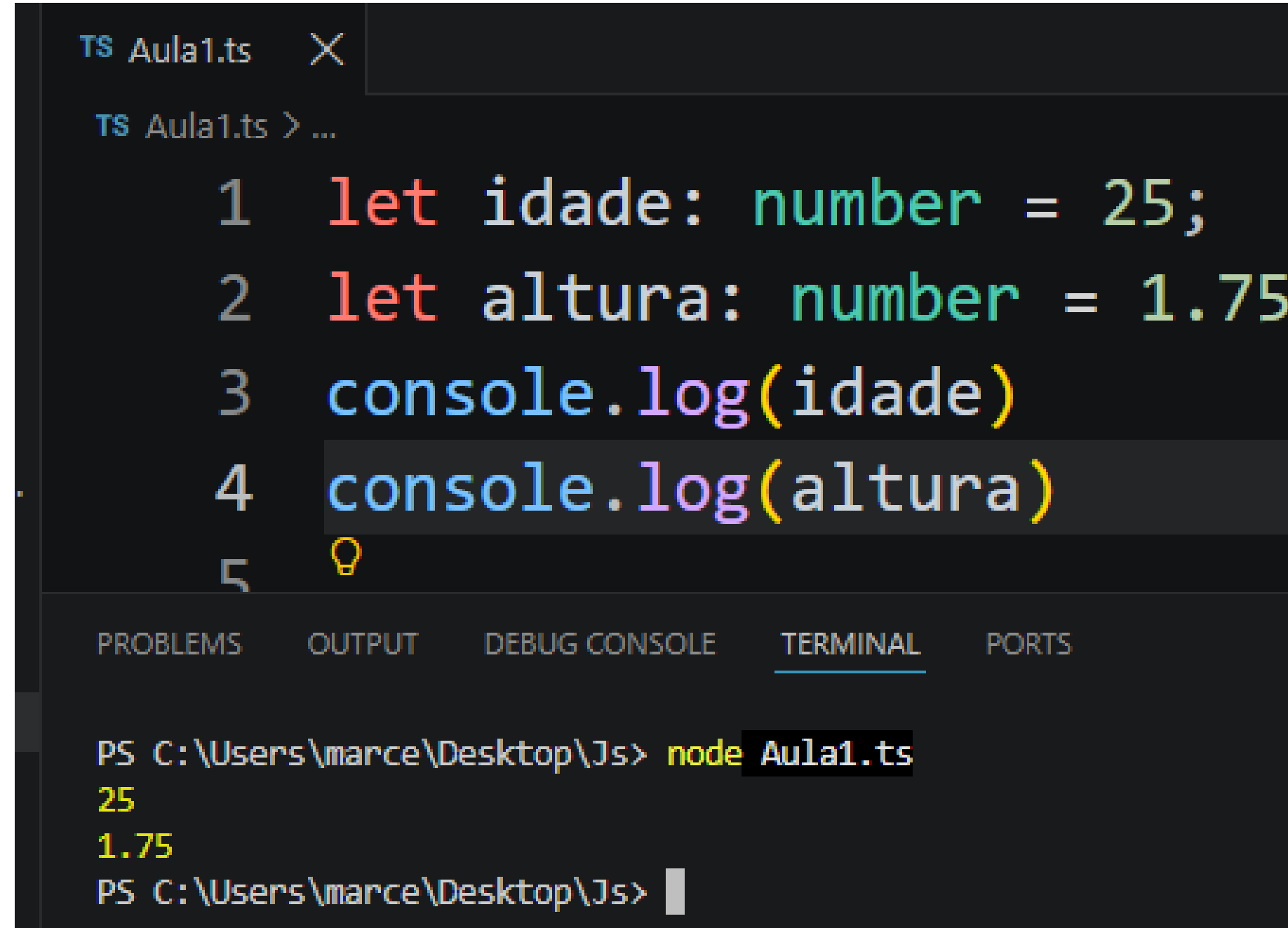
string: usado para armazenar texto.
let nome: string = "Maria";

```
TS Aula1.ts X
TS Aula1.ts > ...
1 let nome: string = "Maria"
2 console.log(nome)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\marce\Desktop\Js> node Aula1.ts
Maria
PS C:\Users\marce\Desktop\Js> 
```

Tipos básicos em TypeScript

number: usado para armazenar números, tanto inteiros quanto decimais, sem distinção entre eles como ocorre em outras linguagens.

A screenshot of a Visual Studio Code editor window. The editor shows a file named 'Aula1.ts' with the following TypeScript code:

```
1 let idade: number = 25;
2 let altura: number = 1.75;
3 console.log(idade)
4 console.log(altura)
5
```

The code is syntax-highlighted. Below the editor, the 'TERMINAL' tab is active, showing the command 'node Aula1.ts' being executed in a PowerShell prompt. The output of the command is displayed as two lines: '25' and '1.75', corresponding to the values of 'idade' and 'altura' respectively.

```
TS Aula1.ts X
TS Aula1.ts > ...
1 let idade: number = 25;
2 let altura: number = 1.75;
3 console.log(idade)
4 console.log(altura)
5
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\marce\Desktop\Js> node Aula1.ts
25
1.75
PS C:\Users\marce\Desktop\Js>
```

ambiente de desenvolvimento TypeScript



Tipos básicos em TypeScript

boolean: usado para armazenar valores verdadeiro ou falso.

TS Aula1.ts X

TS Aula1.ts > ...

```
1 let estaLogado: boolean = true
2 console.log(estaLogado)
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\Users\marce\Desktop\Js> node Aula1.ts

true

PS C:\Users\marce\Desktop\Js> |

ambiente de desenvolvimento TypeScript



Tipos básicos em TypeScript

array: usado para armazenar uma lista de valores do mesmo tipo. Existem duas formas equivalentes de declarar um array tipado:

```
TS Aula1.ts X
TS Aula1.ts > [?] numeros
1 let numeros: number[] = [1, 2, 3]
2 let nomes: Array<string> = ["Ana", "Carlos"]
3 console.log(numeros)
4 console.log(nomes)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\marce\Desktop\Js> node Aula1.ts
[ 1, 2, 3 ]
[ 'Ana', 'Carlos' ]
PS C:\Users\marce\Desktop\Js> [ ]
```

ambiente de desenvolvimento TypeScript



Tipos básicos em TypeScript

any: um tipo especial que permite armazenar qualquer tipo de valor, desabilitando a verificação de tipos naquele ponto do código.

Embora exista, o uso de **any** deve ser evitado sempre que possível, pois anula justamente o principal benefício do TypeScript, que é a segurança da tipagem.

```
TS Aula1.ts X
TS Aula1.ts > ...
1 // Tipo any (evitar quando possível)
2 let qualquerCoisa: any = "pode ser qualquer tipo"
3 console.log(qualquerCoisa)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\marce\Desktop\Js> node Aula1.ts
pode ser qualquer tipo
PS C:\Users\marce\Desktop\Js> 
```



Funções em TypeScript

funcoes em TypeScript

```
1 //Funções Tipadas
2 function somar(a: number, b: number): number {
3     return a + b;
4 }
```

```
5 //Função como expressão (arrow function):
6 const subtrair = (a: number, b: number): number => a - b;
7
```

```
8 //Função sem retorno explícito, usando void:
9 function logMensagem(mensagem: string): void {
10     console.log(mensagem)
11 }
```

O number após os parênteses define o tipo de retorno, e que o TypeScript também consegue inferir esse tipo automaticamente, mas declarar explicitamente é uma boa prática em funções públicas ou reutilizáveis.



TS Aula1.ts X

TS Aula1.ts > ...

```
1 //Parâmetros Opcionais e Valores Padrão
2 //Parâmetros opcionais usam ? e devem vir depois dos obrigatórios:
3 function saudacao(nome: string, sobrenome?: string): string {
4     return sobrenome ? `Olá, ${nome} ${sobrenome}` : `Olá, ${nome}`
5 }
6 console.log(saudacao("Marcelo"))
7 console.log(saudacao("Marcelo", "Silva"))
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\Users\marce\Desktop\Js> node Aula1.ts

Olá, Marcelo

Olá, Marcelo Silva

PS C:\Users\marce\Desktop\Js> █

ambiente de desenvolvimento TypeScript



Funcoes em TypeScript

```
TS Aula1.ts X
TS Aula1.ts > ...
1 //Parâmetros com valor padrão:
2 function calcularDesconto(preco: number, percentual: number = 10): number {
3     return preco - (preco * percentual) / 100
4 }
5 calcularDesconto(100)
6 calcularDesconto(100, 20)
7 //O parâmetro opcional pode ser undefined,
8 // enquanto parâmetro com valor padrão sempre terá um valor definido caso não seja passado.
9 console.log(calcularDesconto(100))
10 console.log(calcularDesconto(100, 20))
11

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\marce\Desktop\Js> node Aula1.ts
90
80
PS C:\Users\marce\Desktop\Js> 
```

ambiente de desenvolvimento TypeScript



funcoes em TypeScript

TS Aula1.ts X

TS Aula1.ts > somarTodos

```
1 //Rest Parameters (5 minutos)
2 function somarTodos(...numeros: number[]): number {
3     return numeros.reduce((total, atual) => total + atual, 0)
4 }
5 console.log(somarTodos(1, 2, 3, 4, 5))
6 //...numeros captura qualquer quantidade de argumentos numéricos em um array,
7 // útil quando não se sabe previamente quantos valores serão passados.
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\marce\Desktop\Js> node Aula1.ts

15

PS C:\Users\marce\Desktop\Js> █

ambiente de desenvolvimento TypeScript



funcoes em TypeScript

TS Aula1.ts X

TS Aula1.ts > ...

```
1 //Tipando Funções como Variáveis
2 //É possível definir o tipo de uma função separadamente
3 // e atribuí-la a uma variável:
4 let operacao: (a: number, b: number) => number
5 operacao = (a, b) => a + b
6 console.log(operacao(5, 3))
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\Users\marce\Desktop\Js> node Aula1.ts

8

PS C:\Users\marce\Desktop\Js> █

ambiente de desenvolvimento TypeScript



funcoes em TypeScript

TS Aula1.ts X

TS Aula1.ts > ...

```
1 //Usando type para nomear o tipo de função:
2 type OperacaoMatematica = (a: number, b: number) => number
3 const multiplicar: OperacaoMatematica = (a, b) => a * b
4 const dividir: OperacaoMatematica = (a, b) => a / b
5 console.log(multiplicar(3,4))
6 console.log(dividir(30,4))
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\Users\marce\Desktop\Js> node Aula1.ts

12

7.5

PS C:\Users\marce\Desktop\Js> █

ambiente de desenvolvimento TypeScript



funcoes em TypeScript

TS Aula1.ts X

TS Aula1.ts > ...

```
1 //Usando type para nomear o tipo de função:
2 type OperacaoMatematica = (a: number, b: number) => number
3 const multiplicar: OperacaoMatematica = (a, b) => a * b
4 const dividir: OperacaoMatematica = (a, b) => a / b
5 console.log(multiplicar(3,4))
6 console.log(dividir(30,4))
7 //Isso é especialmente útil quando funções são passadas
8 // como parâmetros de outras funções (callbacks):
9 function executarOperacao(a: number, b: number, operacao: OperacaoMatematica): number {
10 |   return operacao(a, b);
11 }
12 console.log(executarOperacao(10, 5, multiplicar));
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\marce\Desktop\Js> node Aula1.ts

12

7.5

50

PS C:\Users\marce\Desktop\Js>



Interfaces e Types



ambiente de desenvolvimento TypeScript

Interfaces e Types

Interfaces e types são recursos do TypeScript usados para definir a "forma" que um objeto deve ter, ou seja, quais propriedades ele possui e qual o tipo de cada uma delas.

Isso permite que o próprio código verifique, antes da execução, se um objeto está sendo usado corretamente

(fonte: documentação oficial do TypeScript).

Interfaces e Types

Definindo uma interface: a palavra-chave `interface` é usada para nomear e descrever a estrutura esperada de um objeto.

```
interface Usuario {  
  nome: string;  
  idade: number;  
}
```



ambiente de desenvolvimento TypeScript

Interfaces e Types

Uma vez definida, essa interface pode ser usada para tipar variáveis, garantindo que qualquer objeto atribuído a elas siga exatamente essa estrutura:

```
let usuario: Usuario = {  
  nome: "Ana",  
  idade: 25,  
};
```

```
1 interface Usuario {  
2     nome: string  
3     idade: number  
4 }  
5 let usuario: Usuario = {  
6     nome: "Ana",  
7     idade: 25,  
8 }
```



ambiente de desenvolvimento TypeScript

Interfaces e Types

Interfaces também são muito usadas para tipar os parâmetros de uma função, especialmente quando essa função recebe um objeto como argumento:

```
function apresentar(usuario: Usuario) {  
  console.log(`${usuario.nome} tem ${usuario.idade} anos.`);  
}
```



ambiente de desenvolvimento TypeScript

Interfaces e Types

```
TS Aula1.ts X
TS Aula1.ts > ...
1 interface Usuario {
2   nome: string
3   idade: number
4 }
5 let usuario: Usuario = {
6   nome: "Ana",
7   idade: 25,
8 }
9 function apresentar(usuario: Usuario) {
10   console.log(` ${usuario.nome} tem ${usuario.idade} anos.`)
11 }
12 apresentar(usuario)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\marce\Desktop\Js> node Aula1.ts
Ana tem 25 anos.
PS C:\Users\marce\Desktop\Js> 
```

Interfaces e Types

Além de interface, o TypeScript também permite criar tipos personalizados usando a palavra-chave `type`, que cumpre uma função semelhante:

```
type Usuario = {  
  nome: string;  
  idade: number;  
};
```



ambiente de desenvolvimento TypeScript

Interfaces e Types

Na prática, para descrever a forma de um objeto, interface e type costumam ser intercambiáveis, sendo a escolha entre eles, muitas vezes, uma questão de preferência da equipe ou do projeto.

A principal diferença é que type é mais flexível, podendo representar não apenas objetos, mas também uniões de tipos, enquanto interface é focada especificamente em descrever a estrutura de objetos e classes.

ambiente de desenvolvimento TypeScript

Interfaces e Types

É possível também marcar uma propriedade como opcional, usando ? após o nome dela, indicando que ela não é obrigatória:

TS Aula1.ts > ...

```
1 interface Usuario {  
2     nome: string  
3     idade: number  
4     profissao?: string  
5 }
```

Interfaces e Types

Interfaces são extremamente utilizadas em projetos com React e React Native, principalmente para tipar as props recebidas por um componente, garantindo que ele sempre seja usado com os dados corretos.